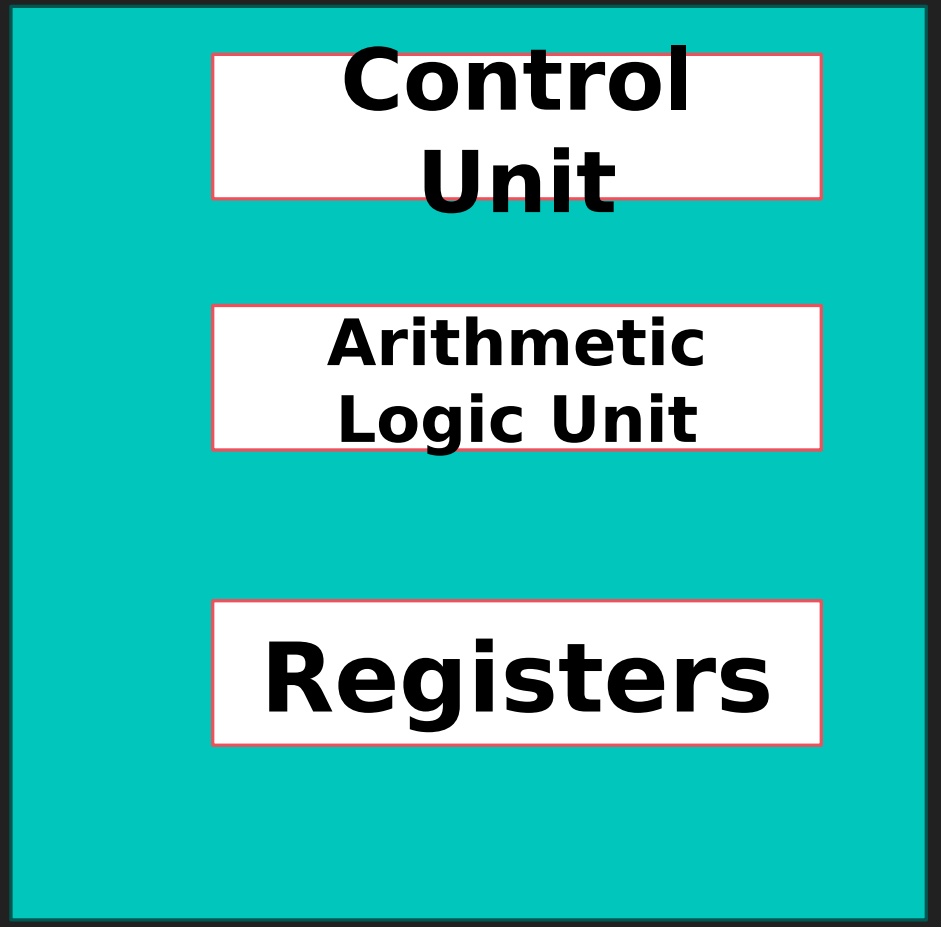
Central Processing Unit(CPU)

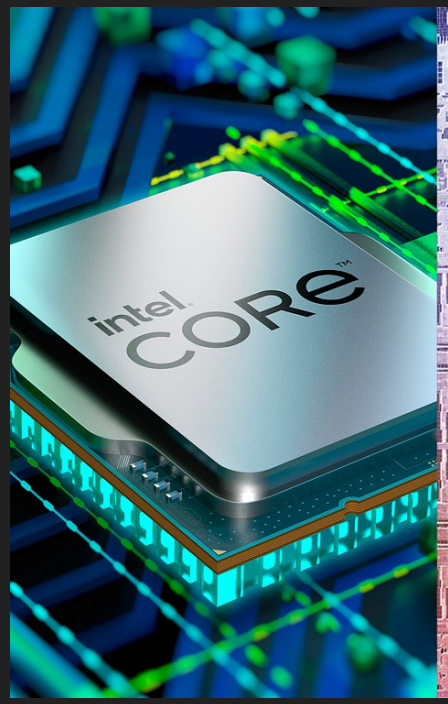
CPU components?

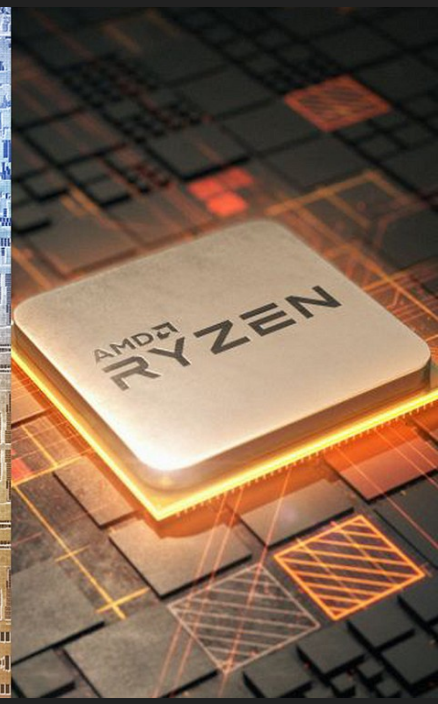
- Control Unit
- Arithmetic Logic Unit
- Registers



CPU organization

- Registers (storage units)
- Arithmetic Logic Unit (ALU)
- Control Unit(CU)





CPU job

- Fetch:
  - Fetching data from memory into CPU register
- Decode
  - The decoding process allows the processor to determine what instruction is to be performed so that the CPU can tell how many operands it needs to fetch in order to perform the instruction
  - The fetched instruction is now decoded for conveying to execution
- Execute
  - Execution in computer and software engineering is the process by which a computer reads and acts on the instructions of a computer program
  - Each instruction of a program is a description of a particular action which must be carried out, in order for a specific problem to be solved

Arithmetic Logic Unit (ALU)

- The arithmetic logic unit (ALU) carries out the logic operations (such as comparisons) and arithmetic operations (such as add or multiply) required during the program execution

Control Unit(CU)

- The control unit is the “policeman” or “traffic manager” of the CPU
- It monitors the execution of all instructions and the transfer of all information
- The control unit extracts instructions from memory, decodes these instructions (making sure data are in the right place at the right time), tells the ALU which registers to use, services interrupts, and turns on the correct circuitry in the ALU for the execution of the desired operation
- The control unit uses a **program counter** register to find the next instruction for execution

Control Unit(CU)

- The control unit is the “policeman” or “traffic manager” of the CPU
- It monitors the execution of all instructions and the transfer of all information
- The control unit extracts instructions from memory, decodes these instructions (making sure data are in the right place at the right time), tells the ALU which registers to use, services interrupts, and turns on the correct circuitry in the ALU for the execution of the desired operation
- The control unit uses a **program counter** register to find the next instruction for execution

CLOCKS

- Every computer contains an internal clock that regulates how quickly instructions can be executed
- The clock also synchronizes all of the components in the system
- The CPU uses this clock to regulate its progress, checking the otherwise unpredictable speed of the digital logic gates
- The clock frequency (sometimes called the clock rate or clock speed) is measured in megahertz (MHz) or gigahertz (GHz)
- For example, an 800MHz machine has a clock cycle time of 1/800,000,000 or 1.25ns
- If a machine has a 2ns cycle time, then it is a 500MHz machine

CLOCKS

$$\text{CPU time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{average cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

CPU registers?

- Registers are a type of computer memory built directly into the processor or CPU (Central Processing Unit) that is used to store and manipulate data during the execution of instructions

- A register is a critical component of computer memory that stores data and instructions for quick processing
- It serves as an efficient temporary storage area where information can be accessed and manipulated quickly in order to carry out complex tasks
- By leveraging the power of registers, modern-day computing systems have become increasingly faster and more reliable than ever before.

CPU registers?

- Registers are a type of computer memory built directly into the processor or CPU (Central Processing Unit) that is used to store and manipulate data during the execution of instructions

- A register is a critical component of computer memory that stores data and instructions for quick processing
- It serves as an efficient temporary storage area where information can be accessed and manipulated quickly in order to carry out complex tasks
- By leveraging the power of registers, modern-day computing systems have become increasingly faster and more reliable than ever before.

What are contents of CPU registers?

Memory Address

Data

Instruction

What is a register is composed of ?

Flip-flops

Register bits?

8 bits

32 bits

16 bits

64 bits

Types of CPU Registers

- **Program Counter (PC)**: The Program Counter keeps track of the memory address of the next instruction to be fetched and executed.
- **Instruction Register (IR)**: The Instruction Register holds the currently fetched instruction being executed.
- **Accumulator (ACC)**: The Accumulator is a general-purpose register used for arithmetic and logical operations. It stores intermediate results during calculations.
- **General-Purpose Registers (R0, R1, R2...)**: These registers are used to store data during calculations and data manipulation. They can be accessed and utilized by the programmer for various purposes.
- **Address Registers (AR)**: Address Registers store memory addresses for data access or for transferring data between different memory locations.

Types of CPU Registers

- **Stack Pointer (SP)**: The Stack Pointer points to the top of the stack, which is a region of memory used for temporary storage during function calls and other operations.
- **Data Registers (DR)**: These registers store data fetched from memory or obtained from input/output operations.
- **Status Register/Flags Register (SR)**: The Status Register or Flags Register contains individual bits that indicate the outcome of operations, such as carry, overflow, zero result, and others. These flags help in making decisions and controlling program flow based on the results of previous operations.
- **Control Registers (CR)**: Control Registers manage various control settings and parameters related to the CPU's operation, such as interrupt handling, memory management, and system configuration

The interaction between registers, Arithmetic Logic Unit(ALU), and Control Unit(CU)

- The CU fetches an instruction from memory and places it into the instruction register
- The CU decodes the instruction to determine the operation to be performed and identifies the registers involved
- The CU issues control signals to select the appropriate registers and routes the data to the ALU
- The ALU performs the arithmetic or logical operation on the data from the selected registers
- The result of the operation is stored back into a register, based on the CU's control signals

RAM Memory

- How does the computer access a memory location corresponding to a particular address?
- We observe that 4M can be expressed as $2^2 \times 2^{20} = 2^{22}$ words.
- The locations for this memory are numbered 0 through $2^{22} - 1$.
- Thus, the memory bus of this system requires at least 22 address lines.
  - The address lines “count” from 0 to $2^{22} - 1$ in binary. Each line is either “on” or “off” collectively indicating the location of the desired memory element.

CPU Registers

- Registers are used in computer systems as places to store a wide variety of data
- A register is a hardware device that stores binary data
- Registers are located on the processor so information can be accessed very quickly
- D flip-flops can be used to implement registers
- One D flip-flop is equivalent to a 1-bit register, so a collection of D flip-flops is necessary to store multi-bit values
- To build a 16-bit register, we need to connect 16 D flip-flops together

Marie simulator registers

Accumulator
AC
16 bits

Memory address register, MAR
12-bit

Memory buffer register MBR
16-bit

Program Counter
PC
12-bit

Instruction register
InREG
16 bits

Input register
IR
12-bit

Output register
OutREG
8-bit

CPU Registers

- At each pulse of the clock, input enters the register and cannot be changed (and thus is stored) until the clock pulses again
- The number of registers in a machine varies from architecture to architecture, but is typically a power of 2, with 16, 32, and 64 being most common
- Registers contain data, addresses, or control information

CPU Registers content

- Data
- Address
- Control information

MARIE CPU REGISTERS

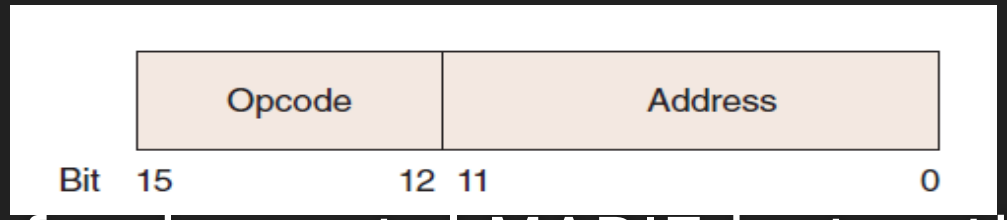
AC(Accumulator)
IR(instruction Register)
MAR(Memory Address Register)
MBR(Memory Buffer Register)
PC(Program Counter)
InREG(Input Register)
OutREG(Output Register)

What is computer’s instruction set architecture (ISA)

- A computer’s instruction set architecture (ISA) specifies the format of its instructions and the primitive operations that the machine can perform
- The ISA is an interface between a computer’s hardware and its software
- Some ISAs include hundreds of different instructions for processing data and controlling program execution

4.7 MARIE (9 of 14)

- This is the format of a MARIE instruction:

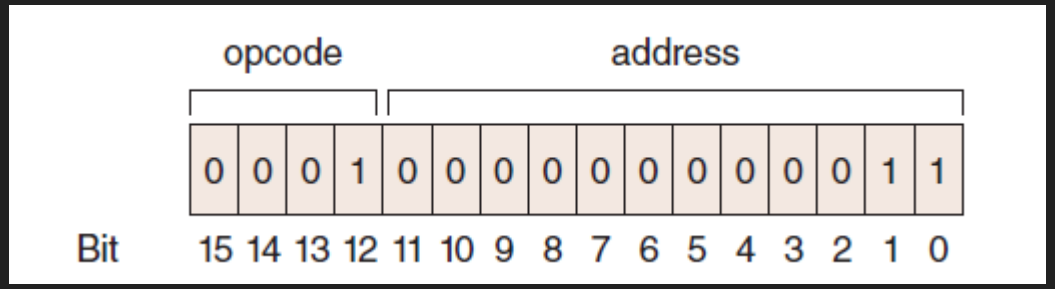


- The fundamental MARIE instructions are:

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load *X*	Load the contents of address X into AC.
0010	2	Store *X*	Store the contents of AC at address X.
0011	3	Add *X*	Add the contents of address X to AC and store the result in AC.
0100	4	Subt *X*	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump *X*	Load the value of X into PC.

4.7 MARIE (10 of 14)

- This is a bit pattern for a **LOAD** instruction as it would appear in the IR:



- We see that the opcode is 1 and the address from which to load the data is 3.

4.7 MARIE (12 of 14)

- Each of our instructions actually consists of a sequence of smaller instructions called *microoperations*
- The exact sequence of microoperations that are carried out by an instruction can be specified using *register transfer language* (RTL).
- In the MARIE RTL, we use the notation M[X] to indicate the actual data value stored in memory location X, and to indicate the transfer of bytes to a register or memory location.

4.7 MARIE (13 of 14)

- The RTL for the LOAD instruction is:

```
MAR ← X
MBR ← M[MAR]
AC ← MBR
```

- Similarly, the RTL for the ADD instruction is:

```
MAR ← X
MBR ← M[MAR]
AC ← AC + MBR
```

A = 2
B = 5
C = 3
TOTAL = A + B – C
C
PRINT(TOTAL)

MARIE Assembler Code Editor

File Edit Assemble Help

```
        LOAD A
        ADD B
        SUBT C
        STORE TOTAL
        OUTPUT
        HALT

A,      DEC 2
B,      DEC 5
C,      DEC 3
TOTAL,  DEC 0
```

MARIE Simulator

File Run Stop Step Breakpoints Symbol Map Help

	label	opcode	operand	hex
000		LOAD	A	1006
001		ADD	B	3007
002		SUBT	C	4008
003		STORE	TOTAL	2009
004		OUTPUT		6000
005		HALT		7000
006	A	DEC	2	0002
007	B	DEC	5	0005
008	C	DEC	3	0003
009	TOTAL	DEC	0	0000

AC 0004 (Hex)
IR 7000 (Hex)
MAR 005 (Hex)
MBR 0004 (Hex)
PC 006 (Hex)
INP... ...

OUTPUT
4
... Control

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	1006	3007	4008	2009	6000	7000	0002	0005	0003	0004	0000	0000	0000	0000	0000	0000
010	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
020	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
030	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
040	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
050	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
060	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
070	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
080	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Machine halted normally.

A = 12
B = 5
C = 3
TOTAL = A + B – C
C
PRINT(TOTAL)

MARIE Assembler Code Editor

File Edit Assemble Help

```
        LOAD A
        ADD B
        SUBT C
        STORE TOTAL
        OUTPUT
        HALT

A,      DEC 12
B,      DEC 5
C,      DEC 3
TOTAL,  DEC 0
```

MARIE Simulator

File Run Stop Step Breakpoints Symbol Map Help

	label	opcode	operand	hex
000		LOAD	A	1006
001		ADD	B	3007
002		SUBT	C	4008
003		STORE	TOTAL	2009
004		OUTPUT		6000
005		HALT		7000
006	A	DEC	10	000A
007	B	DEC	5	0005
008	C	DEC	3	0003
009	TOTAL	DEC	0	0000

AC 000C (Hex)
IR 7000 (Hex)
MAR 005 (Hex)
MBR 000C (Hex)
PC 006 (Hex)
INP...

OUTPUT
12
Control

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	1006	3007	4008	2009	6000	7000	000A	0005	0003	000C	0000	0000	0000	0000	0000	0000
010	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
020	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
030	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
040	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
050	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
060	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
070	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
080	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

TABLE 4.2 MARIE's Instruction Set

Instruction Number		Instruction	Meaning
Bin	**Hex**		
0001	1	Load *X*	Load the contents of address X into AC.
0010	2	Store *X*	Store the contents of AC at address X.
0011	3	Add *X*	Add the contents of address X to AC and store the result in AC.
0100	4	Subt *X*	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump *X*	Load the value of X into PC.